OpenCV Lab Assessment – Five Group Versions

Faculty of Artificial Intelligence & Robotics — Computer Vision Course

Assessment Type: Practical Lab Quiz (5 equivalent versions)

Target Audience: Computer Vision & Robotics students

Total Time: 40 minutes

Total Marks: 20 (MCQ: 14 marks | Written: 6 marks)

Curriculum Scope: Lab 2 — Parts 1 to 9 (Image Geometry, Aspect Ratio, Resizing, ROI, View vs Copy, Padding, Flipping, Translation, Rotation)

Assessment Blueprint (equivalent across all groups)

Q#	Level	Topic	Target skill	Marks
Q1	Medium MCQ	Image Geometry & Memory	shape (H, W, C), uint8, uncompressed byte size	2
Q2	Medium MCQ	Aspect Ratio	computing the new height/width that preserves the ratio	2
Q3	Medium MCQ	Resizing & Interpolation	choosing the right interpolation flag	2
Q4	Medium MCQ	Flipping & Fixed Rotations	flip codes (0, 1, -1) and cv2.rotate codes	2
Q5	Challenging MCQ	ROI: View vs Copy	shared memory vs .copy()	2
Q6	Challenging MCQ	ROI Cropping & Slicing	img[y1:y2, x1:x2] → ROI shape and coordinates	2
Q7	Challenging MCQ	Translation & Rotation Matrix	(x, y) center, dsize = (W, H), tx/ty placement	2
Q8	Written	Function Selection	naming the correct OpenCV function	6

Instructions

1. Solve only the version assigned to your registered group (Group One to Group Five).
2. Each group's pages are self-contained. Do not mix pages between groups.
3. Questions Q1–Q7 are multiple choice. Exactly one option (A, B, C or D) is correct. Circle or write your chosen letter clearly.
4. Question Q8 is written. Write only the exact OpenCV function name in the space provided.
5. The answer key and grading notes are at the end of this document (Instructor copy only).

Examiner / Instructor: _____

Head of Department: _____

OPENCV PRACTICAL ASSESSMENT — GROUP ONE

Total Time: 40 mins | **Total Marks:** 20 | **Coverage:** Lab 2 (Parts 1–9)

Student Name: _____ **Student ID:** _____ **Date:** _____

Q1. [Medium MCQ]

A 3-channel color image is loaded with `cv2.imread('sample.jpg')`. Running `img.shape` returns `(400, 600, 3)` with data type `uint8`.

What are the image height, image width, and the exact uncompressed memory size of the image array in bytes?

A) Height = 600, Width = 400, Memory = 240,000 bytes

B) Height = 400, Width = 600, Memory = 720,000 bytes

C) Height = 400, Width = 600, Memory = 240,000 bytes

D) Height = 600, Width = 400, Memory = 720,000 bytes

Q2. [Medium MCQ]

A camera produces frames of size **1920 × 1080** (width × height). An engineer wants to resize every frame to a **width of 640** while keeping the aspect ratio unchanged.

What new height must be used?

A) 320

B) 360

C) 480

D) 640

Q3. [Medium MCQ]

An engineer needs to downscale a 4K camera frame (3840 × 2160) to a standard resolution (640 × 360).

Which interpolation method is recommended to avoid moiré patterns and aliasing when shrinking images?

A) `cv2.INTER_AREA`

B) `cv2.INTER_NEAREST`

C) `cv2.INTER_CUBIC`

D) `cv2.INTER_LINEAR`

Q4. [Medium MCQ]

A robotic camera is mounted upside-down and captures inverted images. The vision engineer executes:

`frame = cv2.flip(frame, 0)`

Which geometric transformation does this operation perform?

A) Horizontal mirror reflection (left ↔ right)

B) 90-degree clockwise rotation

C) Vertical flip (top ↔ bottom, upside down)

D) Simultaneous horizontal and vertical flip (180-degree rotation)

Q5. [Challenging MCQ]

Analyze the following Python snippet:

```
img = np.zeros((200, 200, 3), dtype=np.uint8)
roi = img[50:150, 50:150]
roi[:] = [0, 255, 0]
print(img[100, 100].tolist())
```

What is the printed output and the reason?

A) `[0, 255, 0]` — slicing creates a memory view, so the original image buffer is modified

B) `[0, 0, 0]` — NumPy slices create independent copies, so `img` is unchanged

C) `[255, 255, 255]` — the assignment fills all channels with white

D) `IndexError` — the coordinate (100, 100) is outside the region of interest

---

Q6. [Challenging MCQ]

An image `img` has shape `(540, 960, 3)`. A student crops a region with:

```
roi = img[100:300, 250:650]
```

What is `roi.shape`, and which pixel of the original image is `roi[0, 0]`?

A) `roi.shape = (400, 200, 3)`; `roi[0, 0]` is `img[250, 100]`

B) `roi.shape = (200, 400, 3)`; `roi[0, 0]` is `img[100, 250]`

C) `roi.shape = (200, 400, 3)`; `roi[0, 0]` is `img[250, 100]`

D) `roi.shape = (400, 200, 3)`; `roi[0, 0]` is `img[100, 250]`

Q7. [Challenging MCQ]

An image `img` has shape `(400, 600, 3)` (height = 400, width = 600). A student wants to rotate it **45° counter-clockwise** around its **geometric center** at **1.0× scale**.

Which option correctly creates the matrix and warps the image?

A) `M = cv2.getRotationMatrix2D((200, 300), 45, 1.0); res = cv2.warpAffine(img, M, (400, 600))`

B) `M = cv2.getRotationMatrix2D((300, 200), -45, 1.0); res = cv2.warpAffine(img, M, (400, 600))`

C) `M = cv2.getRotationMatrix2D((200, 300), 45, 1.0); res = cv2.warpAffine(img, M, (600, 400))`

D) `M = cv2.getRotationMatrix2D((300, 200), 45, 1.0); res = cv2.warpAffine(img, M, (600, 400))`

Q8. [Written — 6 Marks]

An autonomous mobile robot captures rectangular images of size 640 × 480 that must be fed into a neural network expecting a fixed square 640 × 640 input.

Resizing directly squashes the objects and distorts the aspect ratio, and cropping removes peripheral obstacle data. Instead, the perception engineer keeps the aspect ratio and adds **80 pixels of solid black** to the **top and bottom** of every frame, without altering the original content.

Most Appropriate OpenCV Function: _____

OPENCV PRACTICAL ASSESSMENT — GROUP TWO

Total Time: 40 mins | **Total Marks:** 20 | **Coverage:** Lab 2 (Parts 1–9)

Student Name: _____ **Student ID:** _____ **Date:** _____

Q1. [Medium MCQ]

A single-channel grayscale image is read using `img = cv2.imread('scan.png', cv2.IMREAD_GRAYSCALE)`. Running `img.shape` returns `(500, 800)` with data type `uint8`.

What are the image height, image width, and the exact uncompressed memory footprint of this array in bytes?

A) Height = 800, Width = 500, Memory = 400,000 bytes

B) Height = 500, Width = 800, Memory = 1,200,000 bytes

C) Height = 500, Width = 800, Memory = 400,000 bytes

D) Height = 800, Width = 500, Memory = 1,200,000 bytes

Q2. [Medium MCQ]

A camera produces frames of size **1280 × 720** (width × height). An engineer wants to resize every frame to a **width of 320** while keeping the aspect ratio unchanged.

What new height must be used?

A) 180

B) 240

C) 160

D) 320

Q3. [Medium MCQ]

A student needs to enlarge a low-resolution thumbnail (32 × 32) to 256 × 256 for a robotics dashboard. Smooth gradients are prioritized over processing time.

Which interpolation method computes new pixels from a smooth curve through the surrounding 16 neighbors (4 × 4)?

A) `cv2.INTER_NEAREST`

B) `cv2.INTER_CUBIC`

C) `cv2.INTER_AREA`

D) `cv2.INTER_LINEAR`

Q4. [Medium MCQ]

An engineer working on autonomous-driving data augmentation needs to mirror road images **left ↔ right**.

Which call achieves this transformation?

A) `cv2.flip(img, 0)`

B) `cv2.flip(img, -1)`

C) `img[::-1]`

D) `cv2.flip(img, 1)`

Q5. [Challenging MCQ]

Analyze the following Python snippet:

```
img = np.zeros((200, 200, 3), dtype=np.uint8)
img[100, 100] = [120, 120, 120]
roi = img[50:150, 50:150].copy()
roi[:] = [255, 0, 0]
print(img[100, 100].tolist())
```

What is the printed output and why?

A) `[255, 0, 0]` — the ROI slice stays bound to the original array buffer

B) `[0, 0, 0]` — reassigning the ROI resets the surrounding pixels

C) `[120, 120, 120]` — `.copy()` created an independent buffer, so `img` is unchanged

D) `[255, 255, 255]` — the assignment cascades across all channels

Q6. [Challenging MCQ]

An image `img` has shape `(480, 640, 3)`. A student crops a region with:

```
roi = img[60:210, 100:400]
```

What is `roi.shape`, and which pixel of the original image is `roi[0, 0]`?

A) `roi.shape = (300, 150, 3)`; `roi[0, 0]` is `img[60, 100]`

B) `roi.shape = (300, 150, 3)`; `roi[0, 0]` is `img[100, 60]`

C) `roi.shape = (150, 300, 3)`; `roi[0, 0]` is `img[60, 100]`

D) `roi.shape = (150, 300, 3)`; `roi[0, 0]` is `img[100, 60]`

Q7. [Challenging MCQ]

An image `img` has shape `(300, 500, 3)` (height = 300, width = 500). A student wants to rotate the image **90° counter-clockwise** around its **center** and **scale it up by 1.5×**.

Which option correctly invokes `getRotationMatrix2D` and `warpAffine`?

A) `M = cv2.getRotationMatrix2D((150, 250), 90, 1.5); res = cv2.warpAffine(img, M, (300, 500))`

B) `M = cv2.getRotationMatrix2D((250, 150), 90, 1.5); res = cv2.warpAffine(img, M, (500, 300))`

C) `M = cv2.getRotationMatrix2D((250, 150), -90, 1.5); res = cv2.warpAffine(img, M, (500, 300))`

D) `M = cv2.getRotationMatrix2D((250, 150), 90, 1.0); res = cv2.warpAffine(img, M, (500, 300))`

Q8. [Written — 6 Marks]

The overhead camera of a robot arm is mounted sideways, so every frame arrives rotated exactly **90° clockwise**. The controller must restore the upright orientation by rotating each frame exactly **90° counter-clockwise**.

Speed matters: the operation must only rearrange the existing pixels (no interpolation), and the output width and height are swapped.

Most Appropriate OpenCV Function: _____

OPENCV PRACTICAL ASSESSMENT — GROUP THREE

Total Time: 40 mins | **Total Marks:** 20 | **Coverage:** Lab 2 (Parts 1–9)

Student Name: _____ **Student ID:** _____ **Date:** _____

Q1. [Medium MCQ]

A color image is loaded with `img = cv2.imread('robot.jpg')`. Running `img.shape` returns `(300, 700, 3)` with data type `uint8`.

What are the image height, image width, and the total uncompressed memory size of the image array in bytes?

A) Height = 300, Width = 700, Memory = 630,000 bytes

B) Height = 700, Width = 300, Memory = 630,000 bytes

C) Height = 300, Width = 700, Memory = 210,000 bytes

D) Height = 700, Width = 300, Memory = 210,000 bytes

Q2. [Medium MCQ]

An image has size **800 × 600** (width × height). An engineer wants to resize it to a **height of 300** while keeping the aspect ratio unchanged.

What new width must be used?

A) 300

B) 450

C) 400

D) 600

Q3. [Medium MCQ]

In a high-speed robotic tracking system running on an embedded microcontroller, frames must be resized as fast as possible. Visual smoothness is secondary to minimal processing time.

Which interpolation method is the fastest?

A) `cv2.INTER_LANCZOS4`

B) `cv2.INTER_CUBIC`

C) `cv2.INTER_LINEAR`

D) `cv2.INTER_NEAREST`

Q4. [Medium MCQ]

A miniature camera on an underwater vehicle sends frames that are inverted both vertically and horizontally.

Which single call corrects the image along both axes at once?

A) `cv2.flip(frame, 0)`

B) `cv2.flip(frame, -1)`

C) `cv2.flip(frame, 1)`

D) `cv2.rotate(frame, cv2.ROTATE_90_CLOCKWISE)`

Q5. [Challenging MCQ]

Analyze the following Python snippet:

```
img = np.zeros((150, 150, 3), dtype=np.uint8)
patch = img[30:100, 40:110]
patch[:, :] = [0, 0, 255]
print(img[50, 60].tolist())
```

What will be printed?

A) `[0, 0, 255]`

B) `[0, 0, 0]`

C) `[255, 0, 0]`

D) `IndexError: array dimension mismatch`

Q6. [Challenging MCQ]

An image `img` has shape `(600, 800, 3)`. A student crops a region with:

`roi = img[100:400, 200:700]`

What is `roi.shape`, and which pixel of the original image is `roi[0, 0]`?

A) `roi.shape = (300, 500, 3)`; `roi[0, 0]` is `img[100, 200]`

B) `roi.shape = (500, 300, 3)`; `roi[0, 0]` is `img[200, 100]`

C) `roi.shape = (300, 500, 3)`; `roi[0, 0]` is `img[200, 100]`

D) `roi.shape = (500, 300, 3)`; `roi[0, 0]` is `img[100, 200]`

Q7. [Challenging MCQ]

An image `img` has dimensions (height = 400, width = 600). A student wants to translate it **60 pixels to the right (+x)** and **40 pixels downwards (+y)**.

What is the correct matrix `M` and call to `cv2.warpAffine`?

A) `M = np.float32([[1, 0, 40], [0, 1, 60]]); res = cv2.warpAffine(img, M, (400, 600))`

B) `M = np.float32([[1, 0, 60], [0, 1, 40]]); res = cv2.warpAffine(img, M, (600, 400))`

C) `M = np.float32([[1, 0, 60], [0, 1, 40]]); res = cv2.warpAffine(img, M, (400, 600))`

D) `M = np.float32([[0, 1, 60], [1, 0, 40]]); res = cv2.warpAffine(img, M, (600, 400))`

Q8. [Written — 6 Marks]

A drone camera is tilted by an angle that changes from frame to frame. The software needs to straighten each picture by rotating it by an **arbitrary angle** (for example 15°) around the **image center** at **1.0 scale**.

The engineer first needs to build the **2 × 3 transformation matrix** that describes this rotation. The matrix will be passed afterwards to the function that actually warps the image.

Most Appropriate OpenCV Function (the one that creates the matrix):

_____

OPENCV PRACTICAL ASSESSMENT — GROUP FOUR

Total Time: 40 mins | **Total Marks:** 20 | **Coverage:** Lab 2 (Parts 1–9)

Student Name: _____ Student ID: _____ Date: _____

Q1. [Medium MCQ]

An inspection camera generates high-definition color frames with shape `(600, 900, 3)` and data type `uint8`.

What are the image height, image width, and the total uncompressed memory size of the image array in bytes?

A) Height = 900, Width = 600, Memory = 540,000 bytes

B) Height = 600, Width = 900, Memory = 540,000 bytes

C) Height = 900, Width = 600, Memory = 1,620,000 bytes

D) Height = 600, Width = 900, Memory = 1,620,000 bytes

Q2. [Medium MCQ]

A camera produces frames of size **1600 × 900** (width × height). An engineer wants to resize every frame to a **width of 400** while keeping the aspect ratio unchanged.

What new height must be used?

A) 400

B) 225

C) 300

D) 200

Q3. [Medium MCQ]

A student calls `cv2.resize(img, (width, height))` **without** specifying any interpolation argument.

Which interpolation algorithm does OpenCV use by default?

A) `cv2.INTER_LINEAR`

B) `cv2.INTER_NEAREST`

C) `cv2.INTER_CUBIC`

D) `cv2.INTER_AREA`

Q4. [Medium MCQ]

A document-scanning mobile application must rotate an image by exactly **90 degrees counter-clockwise**.

Which call performs this exact right-angle rotation?

A) `cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)`

B) `img[:, ::-1]`

C) `cv2.rotate(img, cv2.ROTATE_90_COUNTERCLOCKWISE)`

D) `cv2.rotate(img, cv2.ROTATE_180)`

Q5. [Challenging MCQ]

Analyze the following Python snippet:

```
img = np.zeros((150, 150, 3), dtype=np.uint8)
img[75, 75] = [200, 200, 200]
crop = img[50:100, 50:100].copy()
crop[:] = [0, 0, 0]
print(img[75, 75].tolist())
```

What will be printed?

A) `[0, 0, 0]`

B) `[255, 255, 255]`

C) `[100, 100, 100]`

D) `[200, 200, 200]`

Q6. [Challenging MCQ]

An image `img` has shape `(540, 960, 3)`. A student crops a region with:

`roi = img[200:350, 400:800]`

What is `roi.shape`, and which pixel of the original image is `roi[0, 0]`?

A) `roi.shape = (400, 150, 3)`; `roi[0, 0]` is `img[400, 200]`

B) `roi.shape = (150, 400, 3)`; `roi[0, 0]` is `img[400, 200]`

C) `roi.shape = (400, 150, 3)`; `roi[0, 0]` is `img[200, 400]`

D) `roi.shape = (150, 400, 3)`; `roi[0, 0]` is `img[200, 400]`

Q7. [Challenging MCQ]

An image `img` has dimensions (height = 500, width = 700). A student wants to translate it **50 pixels to the left (−x)** and **80 pixels downwards (+y)**.

What is the correct matrix `M` and call to `cv2.warpAffine`?

A) `M = np.float32([[1, 0, -50], [0, 1, 80]]); res = cv2.warpAffine(img, M, (700, 500))`

B) `M = np.float32([[1, 0, 80], [0, 1, -50]]); res = cv2.warpAffine(img, M, (700, 500))`

C) `M = np.float32([[1, 0, -50], [0, 1, 80]]); res = cv2.warpAffine(img, M, (500, 700))`

D) `M = np.float32([[0, 1, -50], [1, 0, 80]]); res = cv2.warpAffine(img, M, (500, 700))`

Q8. [Written — 6 Marks]

A car-detection training set contains many photos of cars facing **left**. To create new training examples of cars facing **right** without taking new photos, every image must be mirrored **left ↔ right**.

Only the positions of the pixels change; no pixel value is modified.

Most Appropriate OpenCV Function: _____

OPENCV PRACTICAL ASSESSMENT — GROUP FIVE

Total Time: 40 mins | **Total Marks:** 20 | **Coverage:** Lab 2 (Parts 1–9)

Student Name: _____ **Student ID:** _____ **Date:** _____

Q1. [Medium MCQ]

A computer vision dataset contains color images with shape `(450, 800, 3)` and data type `uint8`.

What are the image height, image width, and the exact uncompressed memory size of a single image array in bytes?

A) Height = 800, Width = 450, Memory = 1,080,000 bytes

B) Height = 450, Width = 800, Memory = 1,080,000 bytes

C) Height = 450, Width = 800, Memory = 360,000 bytes

D) Height = 800, Width = 450, Memory = 360,000 bytes

Q2. [Medium MCQ]

An image has size **1920 × 1080** (width × height). An engineer wants to resize it to a **height of 270** while keeping the aspect ratio unchanged.

What new width must be used?

A) 270

B) 360

C) 640

D) 480

Q3. [Medium MCQ]

A scientific imaging lab needs to enlarge satellite imagery by 8× for terrain inspection. They require the highest-quality (and slowest) interpolation, which uses 64 neighboring pixels (8 × 8) to reduce jagged edges.

Which interpolation flag should they specify?

A) `cv2.INTER_NEAREST`

B) `cv2.INTER_LINEAR`

C) `cv2.INTER_LANCZOS4`

D) `cv2.INTER_AREA`

Q4. [Medium MCQ]

A robot arm picks up square components from a conveyor belt. Components arrive rotated 90 degrees counter-clockwise from their nominal orientation, so the captured frame must be rotated exactly **90 degrees clockwise** to be upright.

Which call does this?

A) `cv2.rotate(frame, cv2.ROTATE_90_CLOCKWISE)`

B) `cv2.rotate(frame, cv2.ROTATE_90_COUNTERCLOCKWISE)`

C) `cv2.rotate(frame, cv2.ROTATE_180)`

D) `cv2.flip(frame, 1)`

Q5. [Challenging MCQ]

Analyze the following Python snippet:

```
img = np.zeros((160, 160, 3), dtype=np.uint8)
sub = img[40:120, 40:120]
sub[:] = [255, 255, 0]
print(img[80, 80].tolist())
```

What will be printed?

A) `[0, 0, 0]`

B) `[255, 255, 0]`

C) `[0, 255, 255]`

D) `[255, 255, 255]`

Q6. [Challenging MCQ]

An image `img` has shape `(720, 1280, 3)`. A student crops a region with:

`roi = img[300:600, 500:900]`

What is `roi.shape`, and which pixel of the original image is `roi[0, 0]`?

A) `roi.shape = (400, 300, 3)`; `roi[0, 0]` is `img[300, 500]`

B) `roi.shape = (300, 400, 3)`; `roi[0, 0]` is `img[300, 500]`

C) `roi.shape = (300, 400, 3)`; `roi[0, 0]` is `img[500, 300]`

D) `roi.shape = (400, 300, 3)`; `roi[0, 0]` is `img[500, 300]`

Q7. [Challenging MCQ]

An image `img` has dimensions (height = 480, width = 640). A vision system needs to rotate it **30° counter-clockwise** around its **center** at **1.0× scale**.

What are the correct arguments for `cv2.getRotationMatrix2D` and `cv2.warpAffine`?

A) `M = cv2.getRotationMatrix2D((240, 320), 30, 1.0); res = cv2.warpAffine(img, M, (480, 640))`

B) `M = cv2.getRotationMatrix2D((320, 240), -30, 1.0); res = cv2.warpAffine(img, M, (640, 480))`

C) `M = cv2.getRotationMatrix2D((320, 240), 30, 1.0); res = cv2.warpAffine(img, M, (640, 480))`

D) `M = cv2.getRotationMatrix2D((240, 320), 30, 1.0); res = cv2.warpAffine(img, M, (640, 480))`

Q8. [Written — 6 Marks]

A satellite viewer stores frames of size 1920 × 1080, but the neural network accepts only inputs of size **480 × 270** (the same aspect ratio).

Every frame must be **shrunk** to exactly one quarter of its width and height before being fed to the network.

Most Appropriate OpenCV Function: _____

INSTRUCTOR COPY — DO NOT DISTRIBUTE TO STUDENTS

Answer Key, Rationales & Grading Notes

Master Quick-Grading Matrix

Group	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8 (expected function)
Group One	B	B	A	C	A	B	D	`cv2.copyMakeBorder()`
Group Two	C	A	B	D	C	C	B	`cv2.rotate()`
Group Three	A	C	D	B	A	A	B	`cv2.getRotationMatrix2D()`
Group Four	D	B	A	C	D	D	A	`cv2.flip()`
Group Five	B	D	C	A	B	B	C	`cv2.resize()`

Q8 grading (6 marks): full marks for the exact function name (the `cv2.` prefix is optional). Partial marks for a closely related function are at the instructor's discretion.

Group One

Q1 — B. `img.shape` is (rows, columns, channels) = (height, width, channels), so height = 400 and width = 600. Memory = 400 × 600 × 3 × 1 byte = 720,000 bytes. *Distractors:* A and D swap height and width; C forgets the 3 channels.

Q2 — B. new_height = old_height × new_width / old_width = 1080 × 640 / 1920 = 360. *Distractors:* 640 (square), 480 (a 4:3 ratio), 320 (half of the width).

Q3 — A. `INTER_AREA` averages all the source pixels under each new pixel, so it is the best choice for shrinking and avoids moiré / aliasing. *Distractors:* NEAREST, LINEAR and CUBIC look at only a few pixels and skip fine detail.

Q4 — C. `flipCode = 0` flips vertically (top ↔ bottom). *Distractors:* A is flipCode 1, D is flipCode -1, B needs `cv2.rotate`.

Q5 — A. Slicing returns a view that shares memory with `img`; writing into `roi` changes `img`. *Distractors:* B assumes slices are copies; D is false because (100, 100) is inside the 200 × 200 array.

Q6 — B. `img[y1:y2, x1:x2]`: rows first. Height = 300 − 100 = 200, width = 650 − 250 = 400, so the shape is (200, 400, 3). `roi[0, 0]` is the original pixel at row y1 = 100, column x1 = 250, i.e. `img[100, 250]`. *Distractors:* swapping rows/columns in the shape (A, D) or in the origin (A, C).

Q7 — D. The center is (x, y) = (width/2, height/2) = (300, 200). A positive angle is counter-clockwise. `dsize` is (width, height) = (600, 400). *Distractors:* A, B, C mix up (y, x) with (x, y), use the wrong sign, or swap `dsize`.

Q8 — `cv2.copyMakeBorder()`. Example: `cv2.copyMakeBorder(img, 80, 80, 0, 0, cv2.BORDER_CONSTANT, value=(0, 0, 0))`.

Group Two

Q1 — C. For a 2D grayscale array, `shape` = (height, width) = (500, 800) with 1 channel. Memory = 500 × 800 × 1 = 400,000 bytes. *Distractors:* A swaps height/width; B assumes 3 channels; D combines both errors.

Q2 — A. new_height = 720 × 320 / 1280 = 180. *Distractors:* 240 (a 4:3 ratio), 160 and 320 (not proportional).

Q3 — B. `INTER_CUBIC` (bicubic) uses a 4 × 4 = 16 pixel neighborhood and gives smooth results when enlarging. *Distractors:* NEAREST gives blocky pixels, LINEAR uses 4 pixels (2 × 2), AREA is meant for shrinking.

Q4 — D. `flipCode = 1` mirrors left ↔ right. *Distractors:* A is a vertical flip; B flips both axes; C (`img[::-1]`) is the NumPy version of a vertical flip.

Q5 — C. `.copy()` creates an independent buffer, so changing `roi` does not affect `img`; the pixel stays `[120, 120, 120]`. *Distractors:* A ignores `.copy()`; B and D describe effects that do not exist.

Q6 — C. Height = 210 − 60 = 150, width = 400 − 100 = 300, so the shape is (150, 300, 3). `roi[0, 0]` is `img[60, 100]` (row y1 = 60, column x1 = 100). *Distractors:* the same row/column swaps as in Group One.

Q7 — B. Center = (250, 150); counter-clockwise = +90; scale = 1.5; `dsize` = (500, 300). *Distractors:* A swaps the center and `dsize`; C uses −90 (clockwise); D keeps the scale at 1.0.

Q8 — `cv2.rotate()`. Example: `cv2.rotate(frame, cv2.ROTATE_90_COUNTERCLOCKWISE)`. Right-angle rotation only rearranges pixels (no interpolation) and swaps width and height.

Group Three

Q1 — A. Height = 300, width = 700. Memory = 300 × 700 × 3 = 630,000 bytes. *Distractors:* B swaps height/width; C and D forget the 3 channels.

Q2 — C. new_width = 800 × 300 / 600 = 400. *Distractors:* 300 (square), 450 and 600 (not proportional).

Q3 — D. `INTER_NEAREST` copies the closest pixel with no averaging, so it is the fastest. *Distractors:* LINEAR, CUBIC and LANCZOS4 average over 2 × 2, 4 × 4 and 8 × 8 neighborhoods respectively.

Q4 — B. `flipCode = -1` flips both axes at once (same as a 180° rotation). *Distractors:* A is vertical only, C is horizontal only, D rotates by 90°.

Q5 — A. `patch` is a view. Pixel (row 50, column 60) lies inside the slice rows 30–100 and columns 40–110, so it becomes `[0, 0, 255]`. *Distractors:* B assumes a copy; C reverses the channel order; D is false because the index is valid.

Q6 — A. Height = 400 − 100 = 300, width = 700 − 200 = 500, so the shape is (300, 500, 3). `roi[0, 0]` is `img[100, 200]`.

Q7 — B. The translation matrix is `[[1, 0, tx], [0, 1, ty]]` with tx = 60 and ty = 40. `dsize` = (width, height) = (600, 400). *Distractors:* A swaps tx/ty and `dsize`; C swaps `dsize`; D breaks the identity part of the matrix.

Q8 — `cv2.getRotationMatrix2D()`. Example: `M = cv2.getRotationMatrix2D((w/2, h/2), 15, 1.0)`; the matrix is then applied with `cv2.warpAffine`.

Group Four

Q1 — D. Height = 600, width = 900. Memory = 600 × 900 × 3 = 1,620,000 bytes. *Distractors:* A and C swap height/width; B forgets the channels.

Q2 — B. new_height = 900 × 400 / 1600 = 225. *Distractors:* 400 (square), 300 (a 4:3 ratio), 200 (not proportional).

Q3 — A. When no interpolation is given, `cv2.resize` uses `cv2.INTER_LINEAR` (bilinear). *Distractors:* the other flags must be requested explicitly.

Q4 — C. `cv2.rotate(img, cv2.ROTATE_90_COUNTERCLOCKWISE)` rotates exactly 90° counter-clockwise. *Distractors:* A rotates clockwise; B (`img[:, ::-1]`) is a horizontal flip; D rotates 180°.

Q5 — D. `.copy()` gives an independent buffer, so `img[75, 75]` stays `[200, 200, 200]`.

Q6 — D. Height = 350 − 200 = 150, width = 800 − 400 = 400, so the shape is (150, 400, 3). `roi[0, 0]` is `img[200, 400]`.

Q7 — A. Left = tx of −50, down = ty of +80, so `M = [[1, 0, -50], [0, 1, 80]]`; `dsize` = (700, 500). *Distractors:* B swaps tx/ty; C swaps `dsize`; D breaks the identity part of the matrix.

Q8 — `cv2.flip()`. Example: `cv2.flip(img, 1)` mirrors left ↔ right.

Group Five

Q1 — B. Height = 450, width = 800. Memory = 450 × 800 × 3 = 1,080,000 bytes. *Distractors:* A swaps height/width; C and D forget the channels.

Q2 — D. new_width = 1920 × 270 / 1080 = 480. *Distractors:* 270 (square), 360 (a 4:3 ratio), 640 (a 4:3 ratio from 480).

Q3 — C. `INTER_LANCZOS4` uses an 8 × 8 neighborhood and gives the highest quality when enlarging. *Distractors:* NEAREST and LINEAR use 1 and 4 pixels; AREA is for shrinking.

Q4 — A. `cv2.rotate(frame, cv2.ROTATE_90_CLOCKWISE)`. *Distractors:* B is the opposite direction, C is 180°, D is a horizontal mirror.

Q5 — B. `sub` is a view; pixel (80, 80) is inside the slice and becomes `[255, 255, 0]`.

Q6 — B. Height = 600 − 300 = 300, width = 900 − 500 = 400, so the shape is (300, 400, 3). `roi[0, 0]` is `img[300, 500]`.

Q7 — C. Center = (width/2, height/2) = (320, 240); counter-clockwise = +30; `dsize` = (640, 480). *Distractors:* A swaps the center and `dsize`; B uses −30 (clockwise); D swaps the center.

Q8 — `cv2.resize()`. Example: `cv2.resize(img, (480, 270), interpolation=cv2.INTER_AREA)` or `cv2.resize(img, None, fx=0.25, fy=0.25, interpolation=cv2.INTER_AREA)`.